

SemanticJoiner: A System for LLM-Powered Semantic Joins

A Project for CS59200-DMA: Data Management for AI

Arpita Saha

Purdue University

West Lafayette, Indiana, USA

saha119@purdue.edu

ABSTRACT

In recent years, the rapid growth of large-scale, unstructured, and multimodal data has made semantic data integration increasingly critical, as many real-world applications require joining records based on high-level semantic relationships rather than exact key equality. Existing approaches to semantic joins often rely on nested-loop or block-based execution using large language models (LLMs), which suffer from high runtime overhead, excessive token consumption, and significant monetary cost. To address these challenges, we propose SemanticJoiner, a novel system for efficient LLM-powered semantic joins that supports arbitrary natural-language join predicates across heterogeneous data. The core idea of SemanticJoiner is to decouple semantic reasoning from exhaustive LLM evaluation by using a locally trained Multi-Layer Perceptron (MLP) model, trained on a small LLM-labeled sample, to prioritize and evaluate candidate tuple pairs at scale. In addition, SemanticJoiner incorporates embedding-based nested loop and hash join strategies as special cases of semantic joins to further improve efficiency. We implemented SemanticJoiner and evaluated it on real-world datasets, including IMDB movie reviews and Amazon product reviews, comparing against state-of-the-art baselines such as LOTUS and ThalamusDB. Our experimental results demonstrate that SemanticJoiner substantially reduces the number of LLM calls and token usage—by up to two orders of magnitude—while achieving competitive or improved accuracy, recall, and F1 scores. These results highlight the promise of hybrid LLM–ML approaches for scalable, cost-efficient semantic query processing.

Note from your instructor: This template is meant to be a sample with a suggested layout, but feel free to change the structure, add/remove sections, or modify the formatting or text to best suit your project.

Feedback from MS1:

- The literature study is good.
- Consider summarizing the methodology in a table for clarity.
- The results are promising.
- The experiments should report on “tokens used” as a key metric maybe.

PVLDB Reference Format:

Arpita Saha. SemanticJoiner: A System for LLM-Powered Semantic Joins
A Project for CS59200-DMA: Data Management for AI.
PVLDB, 14(1): XXX-XXX, 2025.
doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/Arpita33/SemanticJoin>.

1 INTRODUCTION

The rapid growth of large-scale, unstructured, and multimodal data—including text, images, audio, and videos—has created a growing need for expressive data integration primitives that go beyond traditional syntactic joins. In many real-world applications, identifying matching or related items requires reasoning about semantic relationships, such as similarity, contradiction, or sentiment agreement, rather than exact key equality. This motivates the design of semantic join operators that incorporate natural language predicates to perform flexible theta joins over heterogeneous data.

A semantic join enables users to specify join conditions in natural language and retrieve tuple pairs that satisfy high-level semantic constraints. Such an operator is useful across a wide range of applications. For example, in movie analytics, semantic joins can be used to identify films with similar or contradicting sentiments to support analyses such as Oscar nomination trends. In product review datasets, joining products based on sentiment alignment can inform restocking and inventory decisions. In multimodal settings, semantic joins over image–text data can identify relevant keywords for tagging images or videos, enabling effective indexing and keyword-based retrieval.

Despite its expressiveness, designing an efficient semantic join operator remains challenging. Existing work has primarily focused on joinability detection, such as identifying joinable tables or columns, rather than executing joins based on semantic predicates. More recent systems have demonstrated the feasibility of semantic joins using large language models (LLMs). However, these approaches typically rely on nested-loop or block nested-loop joins, resulting in high runtime overhead and significant cost due to large numbers of LLM invocations. Moreover, LLM-based execution is constrained by context length limits, which restrict scalability when operating on large datasets.

In this paper, we introduce SemanticJoiner, an efficient semantic join operator designed to overcome these limitations. SemanticJoiner supports joins over arbitrary data modalities using flexible natural language predicates, while significantly reducing the number of LLM calls required for execution. The key idea is to decouple semantic reasoning from exhaustive LLM evaluation by leveraging a locally trained MLP model to efficiently prioritize candidate matches, thereby enabling scalable execution over large datasets

licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

without sacrificing semantic expressiveness. Our contributions go as follows:

- We propose a novel Semantic Join approach for joining multi-modal data on a Natural Language predicate using a locally trained MLP to reduce LLM calls.
- We conduct a comprehensive experimental evaluation that shows how our method significantly outperforms baselines in terms of execution time and number of LLM calls, thereby reducing the cost of semantic operation.
- We also show the design and implementation of a fast and efficient Semantic Hash Join algorithm based on embeddings using clustering and vector indexing methods.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 details the design and implementation of our proposed algorithm. Section 4 presents our experimental evaluation and discusses the results. Finally, Section 5 concludes the paper and suggests directions for future work.

2 RELATED WORK

In this section, we review prior work in areas relevant to our project.

2.1 Join Discovery and Data Exploration

SEMA-JOIN [3] is an early semantic join discovery approach that identifies joinable columns by mining large web table corpora and modeling semantic relationships using statistical co-occurrence patterns. It focuses on discovering many-to-one semantic joins at the schema level rather than executing joins or evaluating tuple-level semantic predicates.

Chorus [4] addresses large-scale data integration by discovering joinable datasets across heterogeneous sources using scalable indexing and similarity-based techniques. While effective for identifying candidate joins, it primarily targets join discovery and assumes syntactic or weakly semantic compatibility, without supporting natural-language predicates or semantic join execution.

DeepJoin [2] proposes a column-level semantic join discovery method based on transformer-derived embeddings that capture semantic similarity between columns. Although efficient and effective for ranking joinable columns, DeepJoin operates at the column granularity and does not support tuple-level semantic joins or arbitrary natural language predicates.

FreeJoin [8] focuses on scalable join discovery over large data lakes by relaxing strict schema constraints and enabling flexible matching across datasets. It emphasizes efficiency and broad applicability but relies on approximate similarity measures and does not address semantic join execution or LLM-based predicate evaluation.

Approximate query processing techniques, such as Reservoir Sampling over Joins [1], study how to estimate join results efficiently using sampling. While these methods reduce computation cost, they are designed for traditional relational joins and do not address semantic predicates or unstructured data.

2.2 Existing Semantic Join Operators

To our best knowledge, LOTUS [5] is the paper that first demonstrated the implementation of an LLM-powered Semantic Join Operator. In this paper they use a reference algorithm that evaluates a natural-language join predicate over all pairs of tuples from two

input relations. However to reduce the number of LLM calls, they use a proxy sim-filter to determine if embedding similarity between join keys is enough to satisfy the predicate condition. If not then LLM call is invoked. Direct embedding similarity might be weak for some predicates, so Proj-Sim-Filter is used to project the left tuple to a predicted representation of the right hand join key then used to compute embedding similarity. Both compute approximate semantic join to reduce quadratic LLM calls.

[] implements semantic joins as LLM-evaluated theta joins, where the join predicate is specified in natural language and may capture complex semantic relationships such as contradiction, similarity, or complementarity. The baseline implementation follows a tuple nested-loop join, in which the system iterates over all pairs of tuples from two input tables and invokes an LLM once per pair to evaluate whether the natural-language predicate holds. They also implement a block nested-loop join algorithm where they formulate a natural language query to include batches of input tables along with the natural language predicate. The prompt instructs the LLM to identify all matching pairs and outputs the matching indices to reduce token cost.

In DocETL [6], Equijoin is implemented as a semantic equality operator whose reference execution evaluates an LLM-based predicate over all pairs of tuples from two relations, resulting in a quadratic cost. To reduce this overhead, DocETL introduces an approximate Equijoin pipeline that prunes candidate pairs prior to LLM evaluation. The system first computes embeddings for join keys and performs a batched similarity search using automatically calibrated thresholds to meet target recall and precision. Additional lightweight Python filters further eliminate obviously invalid pairs. Only the remaining candidates are verified by the LLM, substantially reducing the number of model invocations while preserving semantic equality semantics.

3 METHODOLOGY

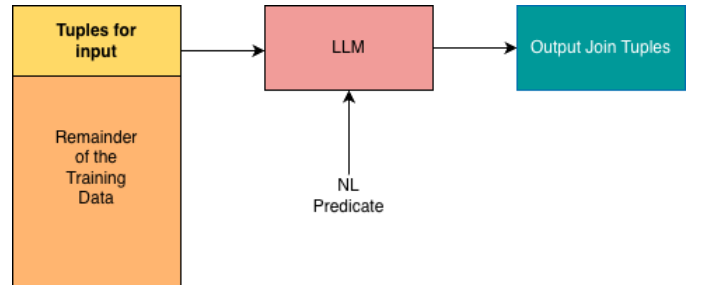


Figure 1: Overview of MLP-based Nested Loop Semantic Join Operator

3.1 MLP-based Semantic Join: Making Semantic Join more efficient

We first take the two tables that we are using to compute the join over. Then we take 10% data and feed it to an LLM (e.g. OPENAI’s GPT-4.1 nano) along with a predicate. We follow the approach from ThalamusDB [7] in this case. Then we take the output of the LLM

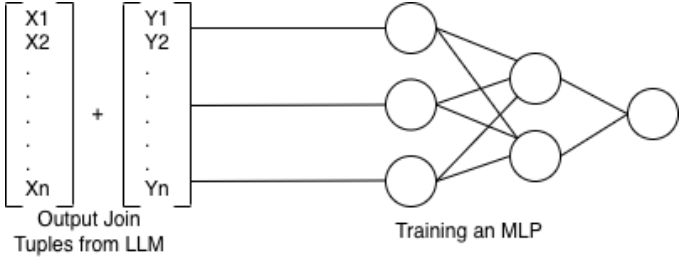


Figure 2: Neural Network architecture for the Multi-Layer Perceptron Model

query and post-process it to produce pairs of rows that should have been output as a join computation, labeling them as positive pairs. We also generate pairs and test if it is present in the positive pair set. If not, it is labeled as a negative pair. Then we use this set of positive and negative pairs to train a Multi-Layer Perceptron neural network model. This simple model learns the relation between the rows of the two tables as defined by the natural language predicate. Then we use the small local model to compute the join over the remaining rows of the two tables. This significantly reduces the #LLM calls as the #LLM calls to process the entire dataset would be much higher.

3.2 Embedding Join: Special Case of Semantic Join

3.2.1 Nested Loop Join. I encoded the text vectors by using a Sentence Transformer Model and then implemented two types of Joins - a basic nested loop join with an outer and inner loop for iterating over the two tables. I also implemented the join by batching the cosine similarity matrix computation. This significantly improved the runtime of the NLJoin Operation. Then I computed the accuracy, precision, recall for Top-K values. As the k value increases, accuracy increases to 100% with improvements in Recall and F1.

3.2.2 Hash Join. Here we implemented a clustering-based approach on embeddings for performing a hash join. We cluster the embeddings of the outer table using K-Means Clustering method. Then we find the cluster centers or centroids. Then we classify the embeddings from the inner table and assign them to the cluster with which it forms the shortest Euclidean distance. This is much faster than traditional nested loop join as it reduces compute of the nested for loops for large tables.

4 EVALUATION

In this section, we evaluate the performance of SemanticJoiner. Our experiments are designed to answer the following questions:

4.1 Experimental Setup

Hardware and Software. All experiments were run on Chameleon Cloud virtual machine. The operating system was [mention OS, e.g., Ubuntu 20.04].

Datasets. We evaluated our Semantic Join algorithm on these datasets:

- The IMovies Dataset: This is the dataset that consists of IMDB movie reviews and their corresponding sentiments. Both the outer and inner tables contain 25K rows.
- Amazon Product Review Dataset: This dataset consists of

Metrics We use Accuracy, Precision, Recall, F1, Tokens used by the LLM (summation of the input and output tokens) and the cost in USD to evaluate the performance of our method against the baselines.

4.2 Results on IMovies Dataset

Table 1 shows the comparison of baseline methods LOTUS and ThalamusDB with our method MLP-based Nested Loop Join. Our method significantly outperforms the block-based join operator’s output from ThalamusDB in all parameters (accuracy, precision, recall, F1, tokens used, cost in USD) except for execution time. This indicates that our approach is more stable and reliable. Since we use both positive and negative pairs of join tuples to train the MLP, it naturally learns the relationship between the embeddings as defined by the natural language predicate. Table 1 also shows that the MLP based nested loop join algorithm has a higher recall than Lotus meaning that it generates fewer false negatives as there are true positives as well as true negatives in the training set.

4.3 Results on Amazon Product Review Dataset

For this dataset, our method significantly outperforms other baselines on all four metrics: Accuracy, Precision, Recall, F1, tokens used and the cost in USD. It also outperforms Lotus in execution time. Thus our method is much better in terms of cost and is also able to capture natural language predicate relations with fewer tokens used.

5 CONCLUSION

In this paper, we introduce an MLP-based nested loop join operator, a novel semantic join operator. Using a small subset of the dataset, we prompt an LLM, instructing it to compute join over a natural language predicate. The output from an LLM is then used to train an MLP locally and use the MLP to calculate join operation over the remaining dataset.

The strengths of our approach are that: our approach scales to substantially larger datasets where prior systems such as LOTUS become impractical, while significantly reducing the monetary cost of join operations. By decoupling correctness from direct reliance on LLM-generated join outputs, it mitigates the unreliability inherent in block-join responses produced by large language models. In particular, even when the LLM generates multiple false positives, the learned MLP ranking model can effectively prioritize true matches by leveraging supervision from both positive pairs returned by the LLM and negative pairs that are absent from its output, leading to improved precision. Finally, the performance of our method depends more strongly on the structure and characteristics of the dataset than on the specific LLM used, enabling more stable and predictable behavior across different model choices. So, our method can overcome the limitations of the existing methods in terms of cost and tokens used by the Large Language Model.

Limitations and Future Work. While our current implementation relies on nested-loop inference for the MLP-based semantic join,

Table 1: Results on the IMDB Movie Review Dataset

Method	Accuracy	Precision	Recall	F1	Execution Time (s)	Tokens Used	Cost (USD)
Lotus	65.16%	99.43%	65.40%	78.90%	51.40	1,487,999	3.68
Block-Join (ThalamusDB)	0.37%	1.52%	0.49%	0.74%	13.68	27,060	0.155
MLP+NLJ	50.22%	51.48%	85.09%	64.15%	24.76	2,533	0.0136
Embedding NLJ ($\tau = 0.4$)	30.16%	61.23%	37.28%	46.35%	0.02	0	0

Table 2: Results on the Amazon Product Review Dataset

Method	Accuracy	Precision	Recall	F1	Execution Time (s)	Tokens Used	Cost (USD)
Lotus	70.8%	38.99%	70.8%	50.28%	52.52	795,211	1.89
Block-Join (ThalamusDB)	8.7%	53.66%	8.8%	15.12%	9.42	11,539	0.0618
MLP+NLJ	77.63%	94.08%	81.58%	87.39%	26.19	1,996	0.01049
Embedding NLJ ($\tau = 0.4$)	4.4%	24.2%	5.1%	8.43%	0.0013	0	0

this step can be further optimized through batching and vectorized computation to improve efficiency. Additional preprocessing techniques such as summarization, thresholding, or clustering could be employed to extract the most relevant textual content prior to MLP inference, reducing redundant comparisons. Beyond text-only joins, incorporating vision-language models (VLMs) would enable semantic joins over multimodal data, allowing the framework to operate on datasets that combine images and text, such as the COCO dataset. Exploring larger or deeper local MLP architectures may further improve matching accuracy, particularly for complex semantic predicates. Finally, we also want to explore the use of the latent embedding from a small LLM after the prefilling phase and use them to do the clustering.

ACKNOWLEDGMENTS

This work was supported by the CS 59200-DMA course at Purdue University. My heartfelt gratitude and thanks to Professor Chunwei Liu for his valuable mentorship, guidance and support throughout this project that brought this work to life.

REFERENCES

- [1] Binyang Dai, Xiao Hu, and Ke Yi. 2024. Reservoir sampling over joins. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26.
- [2] Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. 2022. Deepjoin: Joinable table discovery with pre-trained language models. *arXiv preprint arXiv:2212.07588* (2022).
- [3] Yeye He, Kris Ganjam, and Xu Chu. 2015. Sema-join: joining semantically-related tables using big table corpora. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1358–1369.
- [4] Moe Kayali, Anton Lykov, Ilias Fountalis, Nikolaos Vasiloglou, Dan Olteanu, and Dan Suciu. 2023. CHORUS: foundation models for unified data discovery and exploration. *arXiv preprint arXiv:2306.09610* (2023).
- [5] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2024. Semantic Operators: A Declarative Model for Rich, AI-based Data Processing. *arXiv preprint arXiv:2407.11418* (2024).
- [6] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G Parameswaran, and Eugene Wu. 2024. Docetl: Agentic query rewriting and evaluation for complex document processing. *arXiv preprint arXiv:2410.12189* (2024).
- [7] Immanuel Trummer. 2025. Implementing Semantic Join Operators Efficiently. *arXiv preprint arXiv:2510.08489* (2025).
- [8] Yisu Remy Wang, Max Willsey, and Dan Suciu. 2023. Free join: Unifying worst-case optimal and traditional joins. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–23.